

Research paper

Vision-based obstacle avoidance robotic arm path planning based on a multi-level PPO framework

Qi Sun, Jianhao Guo, Guobing Sun *

The Electronic Engineering College of Heilongjiang University, No. 74 Xuefu Road, Harbin, 150080, China

ARTICLE INFO

Keywords:

Robotic arm
Path planning
Reinforcement learning algorithm proximal
policy optimization
Multi-level framework
RGB-D

ABSTRACT

In complex and dynamic operational environments, there is a growing demand for intelligent robotic manipulators equipped with autonomous path planning and real-time obstacle avoidance capabilities. In response to the sluggish reaction and limited adaptability of conventional path planning algorithms in dynamic environments, this study develops a multi-level path planning algorithm that integrates visual perception with hierarchical motion planning strategies. The proposed algorithm utilizes YOLOv10 to process RGB-D images and construct dynamic 3D occupancy maps, enabling high-precision obstacle recognition within the scene, with an achieved detection accuracy of 99.8%. Subsequently, by integrating with RRT path search and B-spline trajectory smoothing, and through PPO, an end-to-end perception-decision-execution closed-loop path planning is achieved. Experimental results demonstrate that the algorithm achieves path planning success rates of 100% in obstacle-free scenarios, 97.0% with a single static obstacle, 95.67% with multiple static obstacles, and 89.3% in dynamic obstacle environments. These findings validate the algorithm's robustness and adaptability across various typical settings, highlighting its strong generalization capability and potential for practical deployment.

1. Introduction

1.1. Background and motivation

With the rapid advancement of automation and intelligent manufacturing technologies, robotic arms are increasingly undertaking complex tasks across various domains, such as precision assembly in industry, logistics distribution in warehousing, and medical assistance in healthcare. In high-throughput industrial scenarios such as automotive manufacturing, task execution efficiency has become a key indicator for evaluating the deployment value of robotic systems. For example, Banga et al. proposed a cycle time optimization method for resistance spot welding tasks, highlighting the urgent need for path and task efficiency optimization in real-world industrial workflows involving robotic arms [6]. Especially in dynamic or unstructured environments, robotic arms must possess autonomous perception and path planning capabilities to cope with real-time changes in obstacles and environmental constraints [31]. Path planning, as a critical component of the intelligent decision-making system for robotic arms, directly determines the overall efficiency, safety, and level of intelligence of the system. Compared to static environments, path planning in dynamic scenarios faces

greater challenges due to rapid environmental changes and complex obstacle behaviors, imposing higher demands on the system's real-time responsiveness and adaptability [26]. Lei et al. developed a deep reinforcement learning-based path planning approach utilizing Double Q-Networks (DDQN), where the incorporation of tailored reward functions and training mechanisms significantly improved the obstacle avoidance capability and generalization performance of mobile robots operating in unknown dynamic environments [18].

Current path planning algorithms primarily include strategies based on traditional graph search, random sampling, and reinforcement learning. However, each of these methods presents significant limitations when applied to different scenarios [4]. Traditional graph search-based path planning algorithms such as and Dijkstra offer high computational accuracy, but they struggle to meet real-time requirements in complex environments, making them more suitable for static scenarios [8]. Sampling-based methods such as RRT and RRT* are capable of generating paths efficiently; however, their high degree of randomness often leads to suboptimal path efficiency and poor smoothness [4,14]. Deep reinforcement learning (DRL), as an adaptive decision-making strategy [15], has been widely applied in robotic arm navigation and path planning tasks in recent years [5,2]. By interacting with the environ-

* Corresponding author.

E-mail addresses: 2868469363@qq.com (Q. Sun), 1046768443@qq.com (J. Guo), sunguobing@hlju.edu.cn (G. Sun).

<https://doi.org/10.1016/j.rineng.2025.107021>

Received 22 April 2025; Received in revised form 26 August 2025; Accepted 30 August 2025

ment, DRL can learn complex obstacle avoidance strategies; however, it still faces challenges such as high training costs, large sample requirements, significant computational demands, and limited generalization capabilities in dynamic environments, resulting in long training times and difficulties in adapting to complex environmental changes [1].

1.2. Problem description of path planning

In dynamic three-dimensional environments, the core task of robotic arm path planning is defined as finding an optimal trajectory from the starting point to the target point under motion constraints, while ensuring the feasibility and safety of the path [23]. As the degrees of freedom of the robotic arm increase, the dimensionality of its state space expands exponentially, resulting in significant challenges in terms of computational complexity and real-time performance for path planning tasks. Traditional path planning algorithms based on graph search or grid-based methods exhibit satisfactory performance in low-dimensional static environments; however, they become inadequate in high-DoF scenarios due to their reliance on discretized representations and the common assumption of a static environment, which limits their ability to dynamically respond to changes such as moving obstacles [22]. Although sampling-based methods can efficiently generate feasible paths in high-dimensional spaces, the inherent randomness of their sampling process often leads to unstable path quality, necessitating additional smoothing and post-optimization procedures to ensure practical applicability [27]. In dynamic environments, robotic arms must not only perceive the motion states of external obstacles in real time, but also adapt their motion strategies accordingly to ensure obstacle avoidance safety and task execution efficiency [29]. In recent years, deep reinforcement learning (DRL) methods have been introduced into path planning tasks due to their superior policy learning capabilities, enabling adaptive obstacle avoidance through continuous interaction with the environment. However, such methods commonly suffer from lengthy training processes, slow convergence rates, and limited generalization capability, particularly exhibiting instability when encountering unseen environments or complex dynamic changes [3]. Therefore, it is imperative to develop a path planning approach tailored for high-dimensional dynamic environments, equipped with real-time responsiveness, adaptability, and environmental understanding, in order to overcome the deployment limitations of existing strategies in complex scenarios.

1.3. Related work

With the increasing capabilities of robotic systems in multimodal perception, complex environment understanding, autonomous decision-making, equipped with real-time responsiveness and high-degree-of-freedom operations, the challenges of path planning have become increasingly pronounced. These challenges are primarily manifested in the high-dimensional expansion of the state space, heightened environmental uncertainty, and growing demands for real-time responsiveness. In response to these challenges, path planning techniques have progressively evolved from deterministic methods relying on prior rules and static mapping toward intelligent frameworks that integrate data-driven policy learning, autonomous perception, and dynamic environment modeling. At present, robotic path planning approaches can be broadly categorized into three major classes: traditional planning algorithms based on graph search and sampling, reinforcement learning-based methods, and multimodal integrated frameworks that fuse perception and decision-making.

In traditional path planning methods, graph search algorithms such as A* and Dijkstra are widely employed in robot navigation tasks within low-dimensional and structured environments due to their controllable path outputs and capability to compute globally optimal trajectories [17]. To enhance the applicability of such methods in complex environments, Wang et al. proposed the Extended Bidirectional Smoothing A* (EBS-A*) algorithm, which integrates extended distance

computation, bidirectional search, and path smoothing strategies. This approach significantly improves planning efficiency while effectively reducing path angularity and the number of key waypoints [7]. Zhou et al. developed a node occupancy model based on time-window conflict discrimination to optimize the traditional Dijkstra algorithm in multi-task orbital robot scheduling scenarios, effectively mitigating path conflict issues [33]. Graph search-based methods inherently rely on discretized state space modeling, which significantly limits their real-time responsiveness in high-dimensional dynamic scenarios and thus fails to meet the stringent time requirements of complex task planning. Moreover, such algorithms generally assume a static environment and lack mechanisms to respond to dynamic obstacle variations, which severely restricts their applicability in dynamic and unstructured scenarios.

To overcome the aforementioned limitations, researchers have proposed sampling-based path planning methods, such as Rapidly-exploring Random Tree (RRT) and its optimal variant RRT*, to enhance search efficiency and scalability in high-dimensional spaces [37]. Ding et al. proposed an Enhanced Path-expansion Heuristic RRT* (EP-RRT*) algorithm, which incorporates a path-expansion heuristic sampling strategy to improve convergence speed and path quality in narrow or maze-like environments [25]. Gupta et al. proposed an integrated approach that simultaneously optimizes path planning and trajectory tracking. This method was systematically validated in both static and dynamic environments, demonstrating the feasibility of refining traditional strategies for deployment in dynamic scenarios [11]. However, sampling-based methods exhibit inherent randomness in the path generation process and generally suffer from poor global optimality, limited path continuity, and inadequate smoothness. Moreover, such algorithms exhibit delayed responses to dynamic obstacles and lack adaptive mechanisms, making it difficult to meet the real-time performance and robustness requirements of path planning in dynamic environments.

To address the limitations of traditional path planning methods in terms of environmental adaptability and policy generalization, deep reinforcement learning (DRL) algorithms—such as DQN, DDPG, and PPO—have gradually emerged as promising approaches for solving path planning problems in complex environments. Such methods establish an interaction mechanism between the agent and the environment, enabling continuous policy optimization and facilitating the learning of path planning and obstacle avoidance strategies with strong robustness and adaptability. Deep Q-Networks (DQN) are primarily designed for discrete action spaces and are characterized by simple network architecture and stable convergence, making them suitable for low-degree-of-freedom scenarios such as human motion tasks. Gu et al. proposed the DM-DQN architecture, which incorporates an advantage function structure and artificial potential field-based reward design into the traditional DQN framework, thereby enhancing the algorithm's convergence speed and obstacle avoidance capability in both static and dynamic environments [21]. The Deep Deterministic Policy Gradient (DDPG) algorithm, based on the Actor-Critic architecture, is capable of handling continuous action space. However, DDPG algorithm exhibits slow convergence during training, limited robustness to environmental perturbations, and poor policy stability. Therefore, Chen et al. proposed an improved DDPG-based path planning algorithm integrated with Sequential Linear Programming, which guides the policy update process and significantly enhances the stability and success rate of path planning in large-scale dynamic environments [13]. Among policy gradient methods, Proximal Policy Optimization has become one of the most widely used reinforcement learning algorithms in path planning tasks due to its advantages in training stability, sample efficiency, and constrained policy updates. Zhu et al. developed a three-dimensional path obstacle avoidance method based on an improved PPO architecture by integrating a Long Short-Term Memory (LSTM) network and a Collision Prediction Model (CPM), which significantly enhanced the generalization and robustness of autonomous underwater vehicles (AUVs) in dynamic environments [9]. Bhuiyan et al. developed an end-to-end DRL-based path planning system utilizing distance sensor inputs, demonstrating the ef-

effectiveness of observation-decision coupling in industrial robotic arms [38]. Despite the significant advantages of deep reinforcement learning (DRL) methods in policy generation, adaptability, and end-to-end capability, their practical deployment still faces several challenges, including high dependency on training samples, limited generalization to unseen environments, and delayed response caused by the decoupling of perception and decision-making processes.

To further enhance the modeling capability of three-dimensional spatial structures and semantic boundaries, researchers have integrated object detection networks with multimodal visual sensors, such as RGB-D cameras, to construct a unified and fused perception framework. El Ghazouali et al. proposed a method called Fusion Vision, which integrates YOLO with Fast Segment Anything for 3D reconstruction and segmentation of RGB-D images. This approach significantly enhances the accuracy of object boundary extraction and 3D semantic understanding under multi-sensor data conditions [34]. Ozcan et al. proposed a human detection approach that fuses thermal imaging with RGB-D sensor data, significantly improving detection accuracy in complex environments [12]. In robotic manipulation tasks, Yang et al. investigated an RGB-D vision-guided six-degree-of-freedom fueling robotic arm positioning and docking method, which integrates YOLOv8 detection with PnP pose estimation to enhance the accuracy and adaptability of the robotic arm in complex interface recognition and spatial localization tasks [24]. Linder et al. proposed a YOLO-based RGB-D fusion approach, which effectively improving 3D detection accuracy in occluded environments [35]. Takahashi et al. extended the YOLOv3 architecture into the 3D domain and developed a lightweight RGB-D object detection model, which demonstrated enhanced robustness and real-time performance in occluded environments [20]. The integration of object detection with RGB-D perception has significantly advanced environmental mapping and obstacle recognition, providing accurate and efficient perception capabilities for path planning systems. This development establishes a reliable perceptual foundation for subsequent policy generation and trajectory optimization.

Researchers have proposed various integration strategies to address the limitations of single-path planning algorithms in terms of stability, adaptability, and real-time performance, leading to the development of multi-strategy collaborative optimization frameworks for path generation. Josin et al. integrated a backpropagation (BP) neural network into the RRT* framework to develop a hybrid BP-RRT* planning system. By introducing preprocessed spatial partitioning to guide the sampling process, their method significantly improved obstacle avoidance efficiency and path length optimization in collaborative dual-arm robotic path planning tasks [32]. Khan et al. proposed a model-independent trajectory planning and control framework based on the Weighted Jacobian Rapidly-exploring Random Tree (WJRRRT), which enables efficient path planning for both rigid and flexible robotic manipulators [28]. Cui et al. proposed an improved RRT-based behavioral planning method using the PTM algorithm, which constructs a high-order Lipschitz vector field system and incorporates a classification network to enhance path stability and obstacle avoidance robustness [16]. Liang et al. developed a multi-UAV path coordination framework that integrates a CNN-LSTM perception network with a PPO-GIC strategy optimization module, enhancing collaborative perception and obstacle avoidance robustness of multi-agent systems in complex airspace environments [10]. Aghi et al. integrated RGB-D cameras with deep learning models to enable agricultural robots to perceive terrain and obstacles, achieving autonomous navigation in farmland environments [19]. Zhang et al. proposed an autonomous task planning robotic arm system named GPTArm, which integrates a vision-language model (GPT-4V) with real-time perception. This system enables task decomposition, semantic understanding, and multimodal human-robot interaction [36]. The aforementioned studies indicate that path planning strategies based on multimodal fusion are gradually evolving into an integrated perception-planning-control paradigm, offering novel solutions for high-degree-of-freedom robotic arm systems operating in complex environments.

Although deep reinforcement learning has shown potential in high-dimensional path planning, its application in complex and dynamic scenarios remains limited, primarily due to data sensitivity, poor generalization, and insufficient integration with perception. Current methods struggle to handle complex tasks such as six-degree-of-freedom robotic arm operations. To enhance real-time performance and adaptability, it is imperative to develop a closed-loop system that integrates high-precision perception with hierarchical planning for robust and efficient path planning.

1.4. Contributions of the research work

To address the path planning problem of six-degree-of-freedom robotic arms in complex dynamic environments, this study proposes a hierarchical path planning framework that integrates traditional search algorithms, deep reinforcement learning strategies, and RGB-D visual perception. The proposed system integrates top-view RGB-D imagery with a YOLOv10-based dynamic obstacle detection module to enable real-time path adjustment and dynamic construction of a 3D occupancy map. Extensive experiments validate the stability and generalization capability of the method across various obstacle distribution scenarios.

The main contributions of this study are as follows:

1. A hierarchical path planning framework is constructed by integrating A*, RRT, and PPO algorithms: In this framework, the A* algorithm is employed for global path search, leveraging heuristic information to guide macroscopic path planning and enhance overall planning efficiency. The RRT algorithm is utilized for local path refinement, enhancing path exploration capabilities and improving the smoothness of the generated trajectories. The PPO reinforcement learning strategy is employed during the execution phase to enable dynamic adaptation and online optimization. By incorporating a reward shaping mechanism, it enhances adaptability to dynamic environments and improves the robustness of obstacle avoidance.
2. A real-time environmental perception and dynamic obstacle avoidance strategy based on RGB-D imagery and YOLOv10: The system leverages real-time RGB-D imagery to perceive the environment and update obstacle information. YOLOv10 is employed for object detection to locate obstacles, from which depth data within the bounding boxes is extracted to obtain the three-dimensional position and contour of each obstacle. This information is used to update the 3D occupancy map utilized in path planning, thereby enhancing both the efficiency and accuracy of the trajectory generation process.

The remainder of this paper is organized as follows: Section 2 introduces the core path planning and obstacle avoidance methods employed in this study, as well as the dynamic obstacle avoidance mechanism. Section 3 focuses on the design principles of the reward function and outlines the overall workflow of the proposed system. Section 4 describes the configuration of the simulation environment and hardware platform, as well as the procedures for image dataset acquisition and training. Section 5 presents comparative experiments to validate the effectiveness of the proposed approach. Section 6 summarizes the entire study and outlines potential directions for future research.

2. Methods

2.1. Global path planning

A* (A-star) is an optimal path planning algorithm based on heuristic search, which can efficiently compute the optimal path from the start point to the target in a discrete grid environment. Its cost function is defined as:

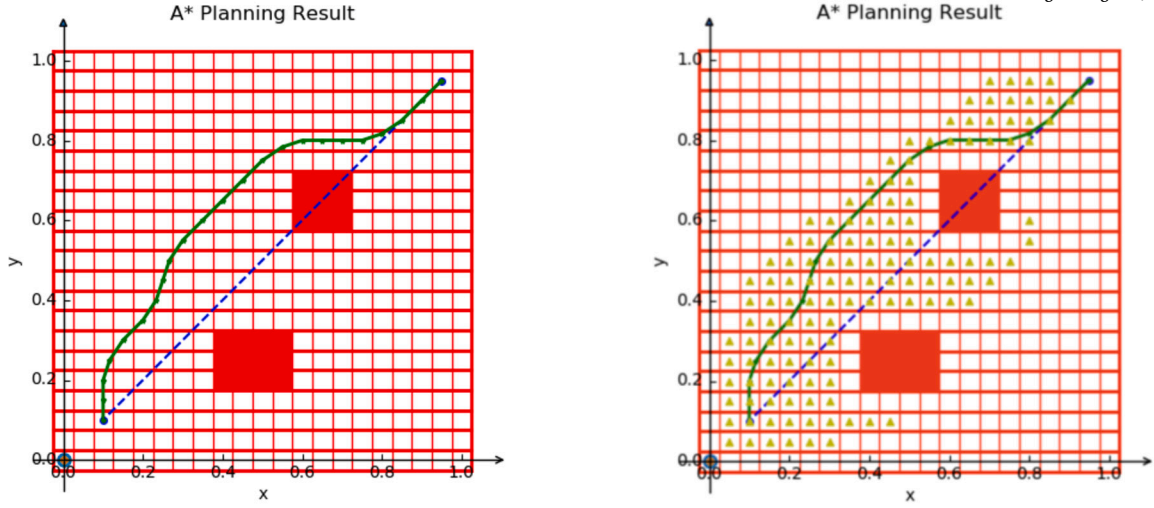


Fig. 1. Schematic diagram of A* algorithm path planning in a 2D grid.

$$f(n) = g(n) + h(n) \quad (1)$$

Where $g(n)$ represents the actual cost from the start node to the current node n , $h(n)$ denotes the heuristic estimate of the cost from node n to the goal (typically calculated using Euclidean or Manhattan distance).

As shown in Fig. 1, the path search result of the A* algorithm in a two-dimensional grid space is illustrated. In the figure, the red grids represent discretized spatial units, the red blocks indicate obstacles, the green curve denotes the final planned path, and the blue dashed line illustrates the heuristic direction (e.g., the straight-line path to the goal). The right figure further illustrates the expanded nodes during the search process (green triangles). A* effectively navigates around obstacles to identify a continuous, cost-optimal path, demonstrating the efficiency and directed nature of its heuristic-based search strategy. In this study, a similar path search process is extended to a three-dimensional discrete space. By downsampling and discretizing the robotic arm's operational workspace into voxel grids, global path planning is efficiently performed in 3D space to generate an initial trajectory map.

2.2. Local path planning

2.2.1. Basic principles of the RRT algorithm

The Rapidly-exploring Random Tree (RRT) algorithm expands a search tree through random sampling, enabling fast exploration of high-dimensional spaces, which makes it particularly suitable for path generation in high-degree-of-freedom robotic arms. Starting from the initial point X_{init} , a random point X_{rand} is sampled, and the nearest node X_{near} in the existing tree is identified. Then, a new node X_{new} is generated from in the direction of X_{rand} with a step size Δt . The computation is given by:

$$X_{new} = X_{near} + \Delta t \cdot \frac{X_{rand} - X_{near}}{\|X_{rand} - X_{near}\|} \quad (2)$$

If X_{new} does not collide with any obstacles, it is added to the search tree and eventually connected to the goal point. Here, Δt denotes the fixed step size.

Fig. 2 illustrates the path generation process of the RRT algorithm in a two-dimensional grid environment. In the figure, the red grids represent the discretized workspace, the red blocks indicate obstacles, the green path denotes the feasible trajectory connecting the start and goal points, and the blue dashed line represents the ideal straight-line reference path between the start and goal. The right figure further illustrates the expanded sampling nodes (light green arrows) during the RRT path search process. It can be observed that RRT explores the free space probabilistically, attempting to avoid obstacle regions while constructing a connected path from the start to the goal. This method exhibits

strong exploratory capabilities and can rapidly generate feasible paths in high-dimensional spaces; however, the resulting paths often suffer from suboptimality and lack smoothness.

2.2.2. B-spline curve smoothing and dynamic modeling constraints

As shown in Fig. 2, although the path successfully avoids obstacles, its pronounced piecewise linear structure and sharp angle changes are unsuitable for robotic arm operation. Therefore, this study incorporates B-spline curves to fit the discrete path output by the RRT algorithm [30]. The mathematical expression is given by:

$$C(u) = \sum_{i=0}^n N_{i,k}(u) P_i, \quad u \in [0, 1] \quad (3)$$

where P_i represents the control points, and $N_{i,k}(u)$ denotes the B-spline basis functions of degree $k-1$, which are computed recursively. To further eliminate sharp turns between nodes, improve the smoothness of the trajectory, and ensure the physical continuity and dynamic accessibility during the operation of the robotic arm.

Simultaneously, to prevent self-collisions during the robotic arm's motion, dynamic modeling constraints are introduced. Let the joint positions of the robotic arm be denoted as x_i , which are required to satisfy:

$$\|x_i - x_j\| \geq d_{min}, \quad \forall i \neq j \quad (4)$$

Here, d_{min} represents the predefined safety distance. This constraint is enforced by incorporating collision detection mechanisms during the RRT search process.

2.3. PPO-based reinforcement learning algorithm

To enhance the adaptability of the robotic arm in dynamic environments, this study employs the Proximal Policy Optimization (PPO) algorithm, a policy optimization method based on the Actor-Critic architecture. The policy network (Actor) generates actions, while the value network (Critic) evaluates the current policy performance, thereby guiding the policy toward improvement. The architecture is illustrated in Fig. 3.

Optimal path adjustment is performed in continuous action spaces, aiming to maximize the cumulative discounted reward.

$$J = \sum_{t=0}^T \gamma^t r(s_t, a_t) \quad (5)$$

Where $r(s_t, a_t)$ is the immediate reward at time t , and $\gamma \in (0, 1]$ is the discount factor.

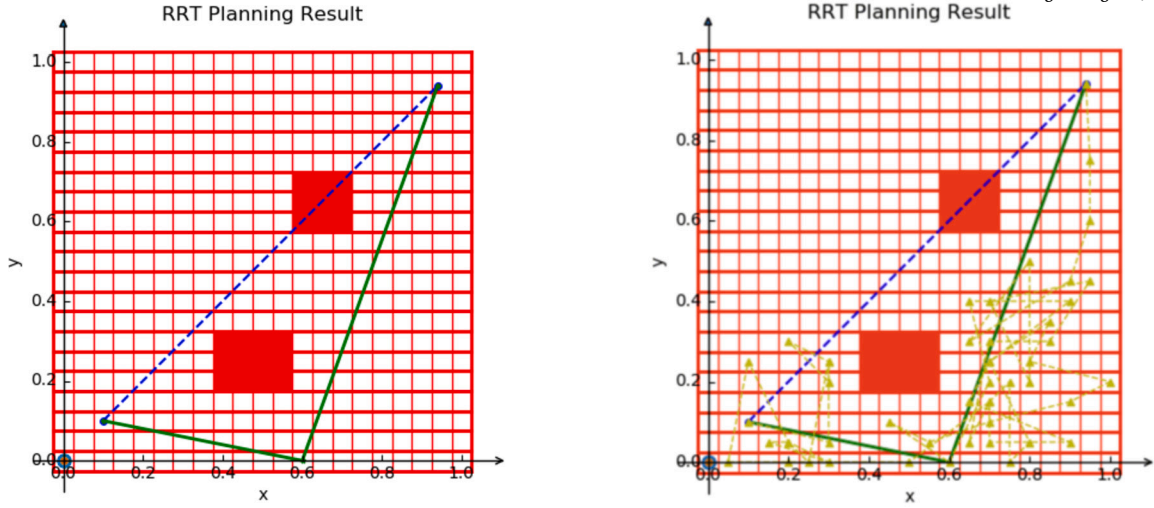


Fig. 2. Schematic diagram of RRT algorithm path planning in a 2D grid.

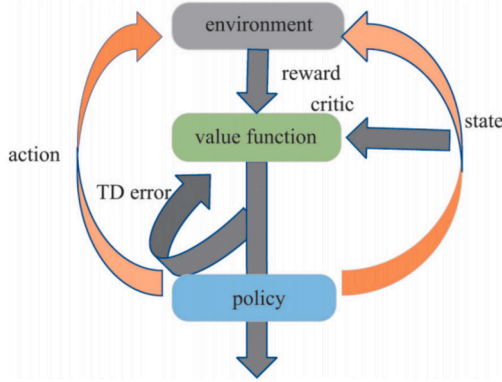


Fig. 3. Actor-Critic Architecture.

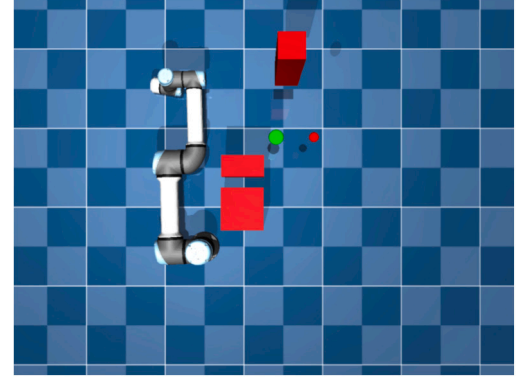


Fig. 4. Top-down RGB-D view.

In this study, the input state S_t of the PPO algorithm represents the environmental perception information of the robot at time t , which mainly includes the position and orientation of the end-effector, the three-dimensional coordinates of the target point, the obstacle distribution obtained from RGB-D perception and YOLOv10 detection, as well as the reference path features generated by A^* and RRT .

During the policy update process, the PPO-Clip method is employed, which uses clipping operations to restrict policy changes, thereby ensuring training stability while avoiding frequent computation of the KL divergence and saving computational resources. The clipped objective function of PPO is defined as:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t [\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)] \quad (6)$$

Where $r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$ represents the policy ratio, $\hat{A}_t$ denotes the advantage estimate, and ϵ is the clipping parameter.

2.4. Dynamic obstacle avoidance mechanism

To enable real-time obstacle avoidance for the robotic arm in a dynamic 3D environment, this study employs an RGB camera to capture top-down view images. Depth images are rendered accordingly and combined to form RGB-D images from a bird's-eye perspective. Obstacle detection is then performed using the YOLOv10 object detection algorithm. While maintaining a lightweight network architecture, the YOLOv10 algorithm incorporates more efficient feature extraction modules and an improved detection head, enabling more accurate object

recognition under limited computational resources. The object detection loss function in YOLOv10 is expressed as:

$$L = L_{\text{cls}} + L_{\text{loc}} + L_{\text{conf}} \quad (7)$$

Where L_{cls} represents the classification loss, L_{loc} denotes the bounding box regression loss, and L_{conf} corresponds to the confidence loss

To ensure the visibility and real-time responsiveness of the path planning system to obstacles, the RGB-D camera is mounted above the robotic arm to continuously capture overhead images and extract the 3D spatial information of obstacles. In the visualization shown in Fig. 4, red indicates obstacles and green represents the target point. The area covered by the RGB camera corresponds to the motion region of the robotic arm during task execution, which refers to the spatial range where the end-effector performs trajectory planning and action execution. This region typically represents a dynamic subset of the arm's full reachable workspace, constrained by task objectives and environmental conditions. In this study, the camera is positioned in a top-down configuration approximately 45° in front of the Joint1 rotation axis of the robotic arm. This placement ensures that the field of view covers the entire target region and obstacle distribution space, as well as the critical operation zone of the end-effector during successful grasping. This configuration guarantees sufficient spatial perception accuracy for path planning, enabling the construction of accurate 3D dynamic occupancy maps and supporting real-time path updates. Although the robotic arm may temporarily move out of the camera's field of view during a few trajectory exploration phases, such states are typically accompanied by significant angular deviations that render the current grasping task in-

feasible. As a result, these instances do not exert a meaningful influence on the path planning training process.

High-precision object detection is performed on the image using the YOLOv10 network, which outputs the category and location of the detected obstacles. After integration, the system generates a dynamic occupancy map and continuously updates the obstacle map in real time. The updated obstacle information is fed back in real time to form a closed-loop mechanism for path planning and obstacle avoidance adjustment. Each frame of perception result is encoded as a local state S_t , which includes information such as the target position and depth image, enabling the path planning policy to make real-time decisions. Compared to static maps, this mechanism offers enhanced real-time responsiveness and environmental adaptability, significantly improving the robustness of obstacle avoidance and the effectiveness of path planning for the robotic arm in complex dynamic scenarios.

3. The proposed algorithm

3.1. Reward function design

The reward function serves as the core of reinforcement learning policy optimization, directly determining the direction in which the agent's behavior is guided during learning. In this study, for the 3D path planning task of a UR5e robotic arm, a multi-component reward function is designed within the PPO framework. The function integrates seven key components: target attraction, obstacle avoidance, motion smoothness, task completion reward, collision penalty, exponential smoothing filter, and initial trajectory-following reward. The reward function is defined as follows:

(1) Goal Attraction Reward R_{goal}

To encourage the robotic arm's end-effector to gradually approach the target point, a reward term inversely proportional to the target distance d_{goal} is defined. A dynamic weight w_g , which varies with the time step, is introduced to regulate the strength of attraction over time:

$$d_{\text{goal}} = \|P_{\text{end}} - P_{\text{goal}}\|_2 \quad (8)$$

$$w_g = \max(0.5, 1 - 0.001 \cdot t) \quad (9)$$

$$R_{\text{goal}} = w_g \cdot (-\log(1 + d_{\text{goal}}) \cdot 10) \quad (10)$$

Where t denotes the current time step, larger values indicate later stages of training, and $0.001t$ is a temporal decay factor that gradually weakens the goal attraction to emphasize obstacle avoidance.

(2) Obstacle Avoidance Reward R_{obstacle}

To improve path robustness in dynamic obstacle scenarios, an obstacle avoidance reward term is designed based on the distance d_{obstacle} between the robotic arm's end-effector and the nearest obstacle:

$$d_{\text{obstacle}} = \|p_{\text{end}} - p_{\text{obstacle}}\|_2 \quad (11)$$

$$R_{\text{obstacle}} = (1 - w_g) \cdot \frac{50}{d_{\text{obstacle}} + \varepsilon}, \quad \varepsilon = 0.1 \quad (12)$$

A larger penalty is applied when the end-effector is close to an obstacle, while the reward approaches zero as the distance increases, thereby reinforcing the safety of the planned path.

(3) Action Smoothness Penalty R_{smooth}

To reduce unnecessary abrupt movements of the robotic arm and improve the executability of the trajectory, an action smoothness term is introduced:

$$R_{\text{smooth}} = -\alpha \cdot \|a_t - a_{t-1}\|^2 \quad (13)$$

Where a_t and a_{t-1} denote the current and previous actions, respectively, and $\alpha = 0.01$ is the penalty coefficient. This term suppresses action oscillations and enhances stability during the execution phase.

(4) Task Success Reward R_{success}

When the distance between the end-effector and the target point falls below a predefined threshold, the task is considered successful. An additional reward is granted, and the current episode is terminated:

$$R_{\text{success}} = \begin{cases} 200, & \text{if } d_{\text{goal}} < 0.02 \\ 0, & \text{otherwise} \end{cases} \quad (14)$$

(5) Collision Penalty $R_{\text{collision}}$

If the distance between the end-effector and an obstacle falls below a predefined safety threshold, it is considered a collision. In such cases, a punitive negative reward is assigned, and the current training episode is terminated:

$$R_{\text{collision}} = \begin{cases} -100, & \text{if } d_{\text{obstacle}} < 0.05 \\ 0, & \text{otherwise} \end{cases} \quad (15)$$

(6) Exponential Smoothing Filter R_{ema}

To improve training stability and mitigate learning instability caused by reward fluctuations, an Exponential Moving Average (EMA) mechanism is introduced to smooth the reward value at each time step:

$$R_{\text{ema}} = \alpha \cdot R + (1 - \alpha) \cdot R_{\text{ema-prev}} \quad (16)$$

Where the smoothing coefficient is set to $\alpha = 0.1$ to enhance training stability.

(7) Initial Path Adherence Reward R_{path}

To encourage the robotic arm to follow the paths generated by A^* and RRT as closely as possible during the planning process—thereby enhancing trajectory quality and stability—an initial path following reward is introduced. This reward is computed based on the minimum distance between the end-effector position and the reference path, ensuring that the agent moves along the predefined optimal trajectory:

$$R_{\text{path}} = -\lambda \min(\|x_{\text{end}} - x_i\|), \quad \forall x_i \in \text{reference path} \quad (17)$$

The complete reward function, integrating all the aforementioned components, is defined as follows:

$$R_t = \max(R_{\text{goal}} + R_{\text{obstacle}} + R_{\text{smooth}} + R_{\text{success}} + R_{\text{collision}} + R_{\text{path}}, -500) \quad (18)$$

and subsequently smoothed using exponential filtering:

$$R_t^{\text{final}} = \alpha \cdot R_t + (1 - \alpha) \cdot R_{t-1}^{\text{ema}} \quad (19)$$

To prevent overfitting during training after designing a well-structured reward function, appropriate termination conditions are established:

1. Noise Annealing Mechanism

In each training episode, a Gaussian noise term is introduced to the policy action outputs to enhance exploration during the early stages of learning. A linear decay strategy is adopted, whereby the Gaussian noise gradually decreases with the training iteration i . As i approaches the maximum number of episodes, the noise is reduced to a minimal level, ensuring that the policy can stably converge to a local optimum

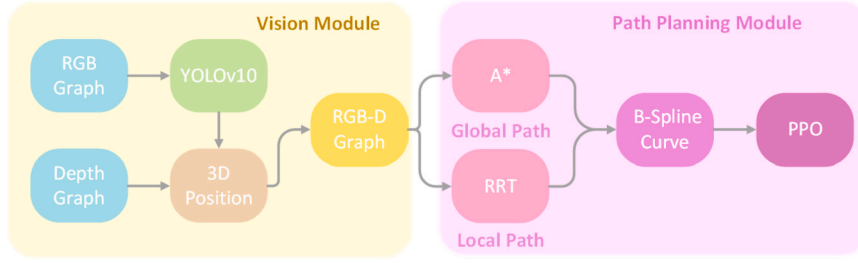


Fig. 5. The workflow of the proposed path planning system.

in the later stages. This approach effectively mitigates the risk of insufficient early-stage exploration and premature convergence to suboptimal solutions.

2. Path Matching Reward Guidance

A path similarity evaluation function is incorporated into the training reward to measure the matching degree between the smoothed reference path and the actual execution trajectory. By comparing the agent's real-time trajectory with the initially planned path, this mechanism provides additional guidance to the policy, reinforcing its learning toward the target path and encouraging the agent to follow the reference trajectory more closely.

3. Termination Criteria and Policy Saving

The system sets a maximum number of training episodes and dynamically saves policy parameters based on cumulative reward and stability conditions. Once the predefined number of episodes is reached or the model converges stably, the training process terminates automatically. The policy model with the best performance is then saved for subsequent experiments.

To enhance the learning efficiency and policy generalization of six-degree-of-freedom robotic arm path planning in 3D dynamic environments, this study introduces a series of optimization strategies during the reinforcement learning training process. These strategies are specifically designed to improve reward stability and training convergence, effectively addressing issues such as policy divergence, overfitting, and oscillation, thereby improving the feasibility and smoothness of the final planned trajectory.

3.2. System workflow

To validate the performance of the proposed multi-level path planning and dynamic obstacle avoidance method in complex 3D environments, a comprehensive experimental pipeline was implemented on the MuJoCo simulation platform. This pipeline encompasses the entire process from environment perception and path generation to trajectory execution, ensuring that the planning strategy can respond in real time to dynamic environmental changes. The experimental workflow is illustrated in Fig. 5.

Firstly, a 3D simulation environment was established in MuJoCo, featuring a UR5e six-degree-of-freedom robotic arm and dynamic obstacles. An RGB camera was positioned in an overhead view to continuously capture real-time RGB-D image data. YOLOv10 network performs object detection on the captured images, which, when combined with the depth map, enables estimation of obstacle positions in 3D space and generates the local environmental state S_t . Subsequently, the path planning system is executed in parallel: the A* algorithm performs global path search, the RRT algorithm refines local trajectories, and the PPO based reinforcement learning module generates dynamic obstacle avoidance strategies based on the environmental state. Together, these three components collaboratively form a comprehensive path planning strategy.

After obtaining the initial trajectory, B-spline interpolation is applied to smooth the path, ensuring the continuity and safety of the robot arm trajectory. The above process is iteratively executed, with the perception module continuously updating the obstacle map in real time, while the planning and control modules adapt the path based on environmental changes, until the PPO policy converges stably and outputs the final smooth and feasible motion trajectory for the robotic arm.

The following presents the pseudocode of the algorithm proposed in this study:

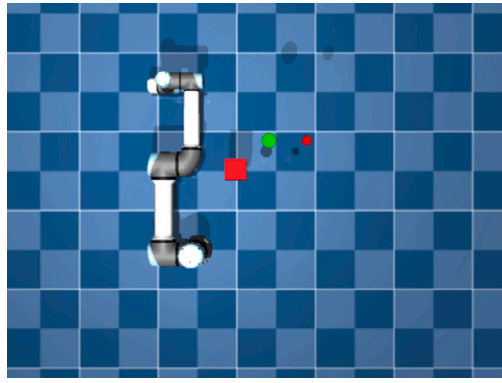
Algorithm 1 Multi-level Path Planning and Execution.

- 1: **Input:** MuJoCo simulation environment, UR5e robot arm, RGBD image stream
- 2: **Output:** Smooth and feasible motion trajectory for the robot arm
- 3: Initialize the simulation environment, RGBD camera, UR5e arm, and obstacles
- 4: **while** training not completed **do**
- 5: Capture RGB and depth images, perform object detection using YOLOv10
- 6: Fuse detection results with depth to estimate 3D obstacle positions
- 7: Encode current environment state S_t as input to planning and PPO modules
- 8: Use A* for global path guidance and RRT for local path refinement
- 9: PPO selects actions based on S_t , fused with planning results for execution
- 10: Evaluate reward R_t based on smoothness, obstacle avoidance, and goal reaching
- 11: Update PPO policy network accordingly
- 12: **if** environment changes **then**
- 13: Update obstacle map and replan the path
- 14: **end if**
- 15: **end while**
- 16: Generate the final planned path with the converged PPO policy
- 17: Apply B-spline interpolation to smooth the final path (post-processing)
- 18: Output the final policy and smoothed trajectory

4. Experimental setup

4.1. Simulation environment and hardware platform

This study utilizes the MuJoCo simulation platform to construct a three-dimensional workspace, simulating the motion of a six-degree-of-freedom robotic arm in complex environments. The platform offers efficient dynamic modeling and precise physical simulation capabilities, supporting key tasks such as robotic arm path planning, collision detection, and trajectory optimization. It accurately simulates the interaction between the robotic arm and obstacles. The experiment employs the UR5e robotic arm model, incorporating dynamic modeling constraints to ensure that all joints and links maintain a minimum safety distance. This prevents occurrences such as joint limit violations, self-collisions, and end-effector flipping during path planning. Joint angle limits are defined to guarantee safe operation, as detailed in Table 1.



(a) RGB Image



(b) Depth Map

Fig. 6. Schematic diagram of RGB and Depth image fusion for constructing RGB-D input.

Table 1

Joint Angle Limits of the UR5e Robotic Arm (Collision Avoidance Constraints).

Joint No.	Joint Name	Minimum Angle (°)	Maximum Angle (°)
Joint 1	Base	-180	+180
Joint 2	Shoulder	-90	+90
Joint 3	Elbow	-90	+90
Joint 4	Wrist 1	-180	+180
Joint 5	Wrist 2	-120	+120
Joint 6	Wrist 3	-180	+180

Table 2

Hardware Configuration.

Parameter	Specification
CPU	Intel® Core™ i9-13900 K
Memory	64 GB
GPU	NVIDIA RTX 4090
Operating System	Ubuntu 22.04 LTS
Simulation Platform	MuJoCo 2.1.0

Table 3

YOLOv10 Training Performance.

Model	P	R	mAP50	mAP50-95
YOLOv10	0.998	1	0.995	0.845

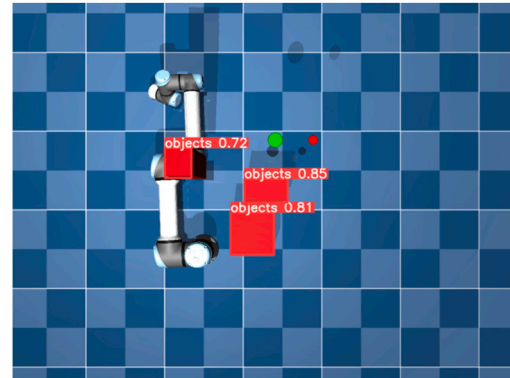


Fig. 7. Detection Bounding Boxes Visualization.

The basic hardware environment Settings of this study are shown in Table 2

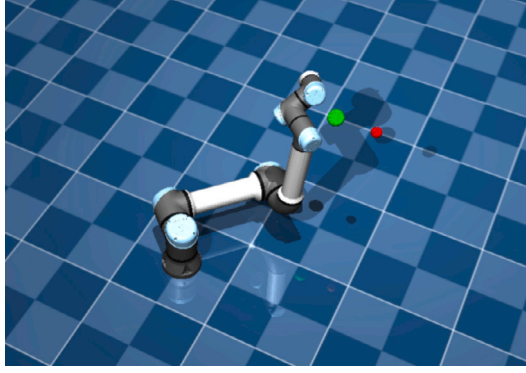
4.2. Image dataset acquisition and training

To enable real-time dynamic obstacle detection, this study employs an RGB camera to capture top-down images of the robotic arm's workspace. Corresponding depth images are acquired through the rendering interface of a virtual camera. In the simulation environment, an RGB camera is fixed above the robotic arm to ensure full coverage of its operational workspace. One or more dynamic and static obstacles of different colors are placed within the environment. Real-time RGB and depth images are acquired using the `mj.mjr_readPixels` interface via simulation rendering. The collected images are preprocessed and stored in standardized formats. Subsequently, the RGB images are concatenated with the normalized depth images to form four-channel RGB-D data, which is then used for training. A total of 5,000 RGB-D images were collected using the overhead camera, with obstacles placed within the reachable workspace of the robotic arm. The acquired images were manually annotated, and the dataset was split into training, validation, and testing sets in an 8:1:1 ratio—resulting in 4,000 images for training, 500 for validation, and 500 for testing. The training results are presented in Table 3, showing a Precision of 99.8% and a Recall of

100%. These results demonstrate the system's high efficiency and accuracy in detecting and localizing targets within the scene.

As illustrated in Fig. 6, the scene involves multiple obstacles: (a) shows the RGB image captured by a fixed overhead camera, while (b) presents the depth image generated in the simulation, which provides height information of the obstacles. After normalization, the two images are concatenated to form the RGB-D image required for this study.

In this study, the YOLOv10 object detection algorithm is used to train on the collected image dataset for obstacle recognition. The trained YOLOv10 model is then deployed within the simulation system to perform real-time processing on RGB images, outputting 2D bounding boxes along with the class and positional information of detected targets. An example of the detection result is shown in Fig. 7. To achieve dynamic obstacle avoidance in 3D space, each pixel coordinate within the bounding box is mapped to the corresponding depth information, allowing the estimation of the object's actual position and volumetric contour in 3D space. A real-time 3D obstacle occupancy map is then constructed and continuously updated. This 3D obstacle information forms part of the current environment state and is input to the path planning module, enabling dynamic path adjustment and optimized avoidance strategies.



(a) Simulation Diagram



(b) Reward Curve Plot

Fig. 8. No Obstacle Scenario.

5. Experimental results and comparative analysis

To evaluate the adaptability of the proposed method under various scenarios, four types of experimental environments are designed:

1. No Obstacle Scenario

Fig. 8 illustrates the path planning experiment conducted in an ideal environment without obstacles. In this scenario, the robotic arm operates without any external interference, completing the basic motion from the starting point to the target. This setup is used to evaluate the algorithm's convergence performance and path generation capability under unconstrained conditions. It can be observed that the algorithm rapidly completes policy learning during the early training phase, with the reward values increasing quickly and stabilizing thereafter. Both the raw and doubly smoothed reward curves demonstrate excellent convergence and stability. The method efficiently accomplishes the path planning task and consistently outputs stable actions, achieving a path planning success rate of 100%.

2. Single Static Obstacle

As shown in Fig. 9, the single static obstacle scenario involves a fixed-position obstacle. The training of the obstacle avoidance strategy in this setup is used to verify the feasibility and accuracy of the generated path. From the reward curve, it can be observed that the algorithm quickly achieves a high reward in the early stages, with minimal overall fluctuation, indicating good convergence and stability. The path planning success rate reached 97%.

3. Multiple Static Obstacles

As shown in Fig. 10, the presence of multiple static obstacles increases the environmental complexity. It can be observed that although the algorithm converges more slowly, it still achieves a high reward value and converges stably, demonstrating good stability and effectiveness. The path planning success rate reaches 95.67%.

4. Dynamic Obstacles

As shown in Fig. 11, the dynamic obstacle scenario involves the introduction of moving obstacles. By integrating RGB-D images with the YOLOv10 detection algorithm, the system achieves real-time obstacle detection and localization. The detection results are fed back to the reinforcement learning module to assist in path decision-making. Under these conditions, the proposed framework achieves a path planning success rate of 89.3%, demonstrating its effectiveness in handling

real-time dynamic environments. This scenario closely simulates real-world robotic working environments. Despite the continuous positional changes and disturbances caused by moving obstacles, the algorithm demonstrates strong adaptability and is ultimately capable of reliably generating collision-free paths.

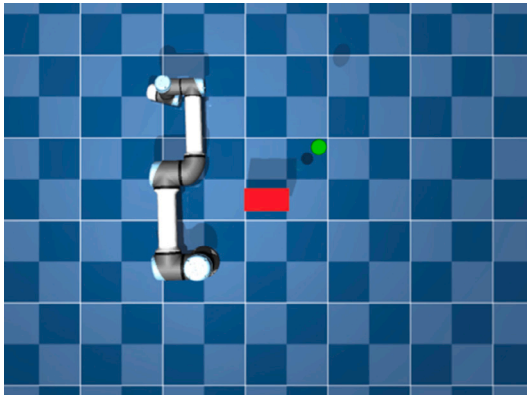
To demonstrate the superiority of the proposed algorithm, we performed comparative experiments in a dynamic obstacle environment, benchmarking against the traditional PPO, the reward-function-modified PPO, and the PPO algorithm enhanced with A* and RRT integration, as shown in Fig. 12.

As shown in Fig. 12(a), the traditional PPO algorithm exhibits considerable fluctuations during training. Although it achieves relatively high rewards in the early stages, the policy tends to degrade as training progresses, with a declining reward trend, indicating poor robustness in dynamic environments. Fig. 12(b) illustrates the PPO algorithm with the improved reward function, demonstrating enhanced overall stability of training rewards. The policy exhibits greater robustness in adapting to environmental changes, thereby validating the effectiveness of the reward function design in this study. Fig. 12(c) presents the training results of the PPO algorithm proposed in this study, which integrates A* and RRT path priors. It is evident from the figure that the proposed algorithm rapidly converges despite interference from dynamic obstacles, maintaining high and stable reward values throughout the training process with significantly reduced fluctuations, demonstrating strong environmental adaptability and optimal planning performance. These results indicate that incorporating prior path planning information effectively guides policy learning, significantly enhancing training efficiency and policy quality when confronting highly dynamic and complex scenarios.

After integrating A* global search with RRT local optimization and applying B-spline curve smoothing, the joint angle variation trends of the six robotic arm joints during path planning were compared before and after smoothing. The results are presented in Fig. 13.

Compared to the joint angle variations in the unoptimized path shown in Fig. 13(a), the curves in Fig. 13(b) exhibit greater smoothness and continuity, with significantly reduced oscillations, particularly pronounced in the shoulder link and forearm link joints. The optimized trajectory becomes more continuous and smooth, with joint angle variations exhibiting gentler transitions. The integration of A* and RRT path planning results, followed by B-spline smoothing, effectively enhances the overall executability and smoothness of the trajectory, reduces abrupt joint angle changes and unnecessary adjustments during the robotic arm's motion, and improves the dynamic trackability of the path.

Experimental results across various scenarios demonstrate that the proposed path planning system exhibits robust stability and strong generalization capabilities, maintaining a success rate of 89.3% even in dy-

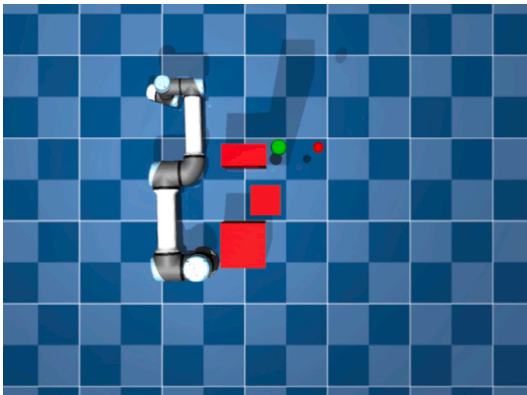


(a) Simulation Diagram



(b) Reward Curve Plot

Fig. 9. Single Static Obstacle.

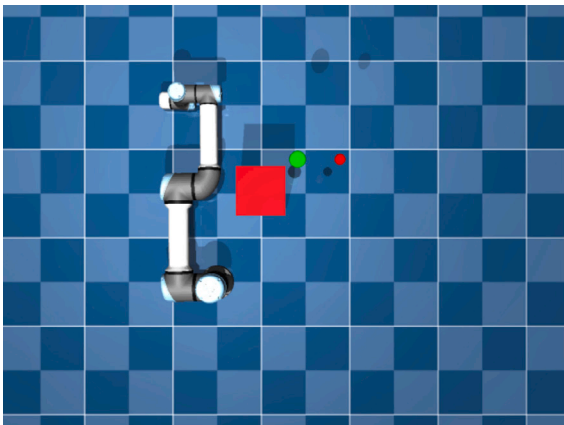


(a) Simulation Diagram

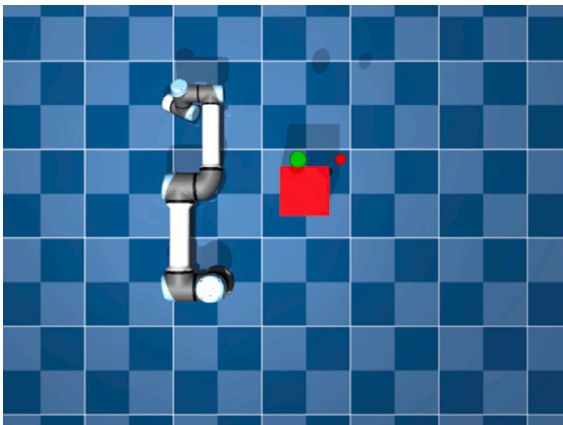


(b) Reward Curve Plot

Fig. 10. Multiple Static Obstacles.

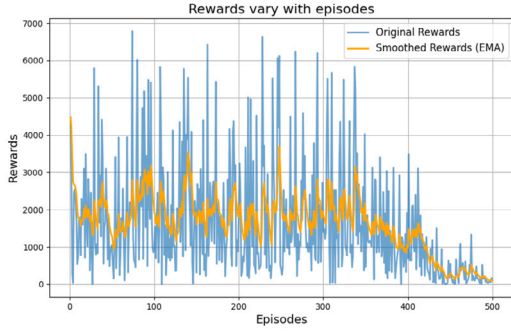


(a) Illustration Before Obstacle Movement



(b) Illustration After Obstacle Movement

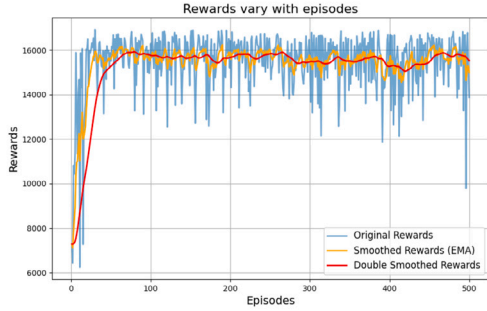
Fig. 11. Dynamic Obstacles.



(a) Reward Curve of the Traditional PPO Algorithm



(b) Reward Curve of the PPO Algorithm with Modified Reward Function



(c) Reward Curve of the PPO Algorithm Integrating A* and RRT

Fig. 12. Comparison of Reward Function Variation Curves under Different PPO Algorithm Architectures.

dynamic obstacle environments, thereby demonstrating effective real-time path reconstruction and local adjustment abilities. Compared to the traditional PPO strategy, which often suffers from convergence oscillations and policy degradation in dynamic scenarios, the multi-level approach integrating A*, RRT, and PPO proposed in this study enhances policy stability and accelerates reward convergence, thereby validating the effectiveness of combining path priors with policy learning. Compared with the PPO algorithm incorporating an improved reward function, which shows enhancements in convergence efficiency and oscillation control, the integrated PPO strategy proposed herein demonstrates superior robustness and path feasibility in performance. The joint angle trajectories of the robotic arm relative to the end-effector further validate the critical role of B-spline smoothing in enhancing path feasibility and continuity during execution. The optimized trajectory significantly reduces abrupt angle changes across the six joints, thereby improving the actuator's tracking accuracy of the planned path. Moreover, the proposed method

exhibits some performance degradation in high-density multi-obstacle scenarios, primarily due to the accumulation of YOLOv10 detection errors, which lead to delayed updates in the occupancy grid and local oscillations in collision recovery paths. This indicates that there remains room for improvement in managing perception errors. The proposed algorithm outperforms existing DRL-based path planning methods in terms of planning efficiency, policy stability, and trajectory smoothness, demonstrating superior adaptability in challenging scenarios. However, future work should focus on enhancing perception accuracy and local obstacle recovery strategies to further improve the system's overall environmental adaptability.

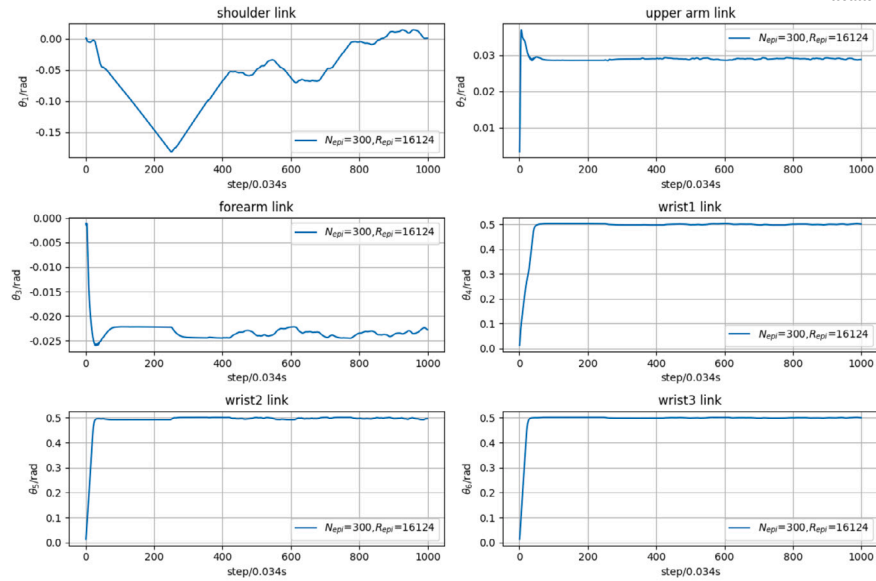
6. Conclusion

This study addresses the challenges of insufficient robustness in path planning and delayed perception response for six-degree-of-freedom robotic arms operating in complex dynamic environments, by proposing an intelligent path planning system that integrates multi-level path search strategies with visual perception mechanisms. The system utilizes YOLOv10 for object detection to obtain obstacle location information from RGB images, achieving a detection accuracy of up to 99.8%. Combined with depth maps, it constructs a three-dimensional occupancy grid for dynamic obstacles, enabling high-precision modeling and real-time updates. Based on the output images, the path planning task is conducted using parallel execution of A* and RRT algorithms to enhance the globality and diversity of path search. B-spline curves are introduced to smooth the paths, thereby improving path feasibility and smoothness.

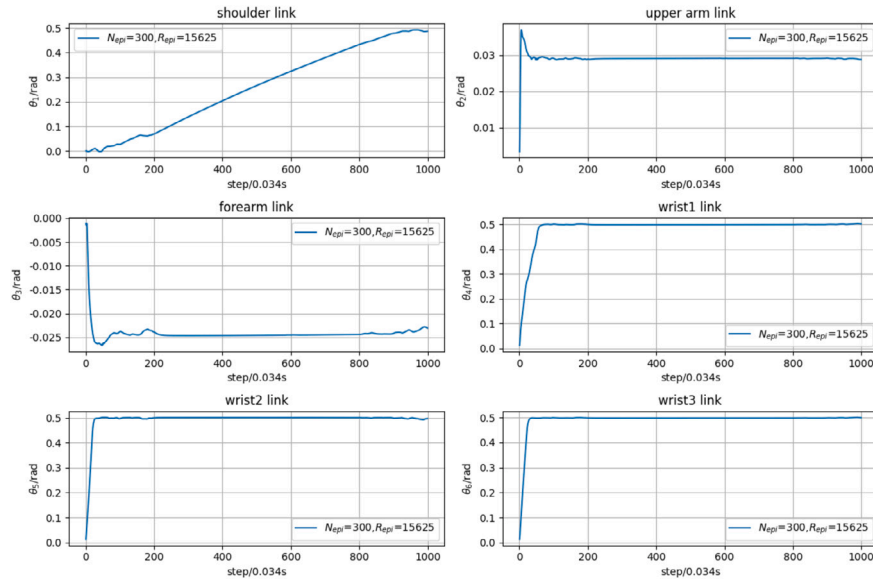
Experimental results demonstrate that the proposed method exhibits strong path planning capabilities across scenarios involving no obstacles, single obstacles, multiple obstacles, and dynamic obstacles. The system maintains high path planning success rates and obstacle avoidance stability, achieving success rates of 100%, 97%, 95.67%, and 89.3% respectively in these scenarios. Notably, in dynamic obstacle scenarios, the PPO algorithm integrating planning priors and perceptual information demonstrates significant improvements over the traditional PPO and reward-modified PPO algorithms in terms of convergence speed, trajectory stability, and final reward values, exhibiting enhanced adaptability and generalization capabilities.

Compared to methods relying solely on a single search or learning strategy, this study addresses the generalization limitations of reinforcement learning in unseen scenarios by synergistically integrating classical path planning algorithms with reinforcement learning mechanisms, thereby enhancing both the global optimality and robustness of path search. Moreover, in scenarios with high-density obstacle occlusion or significant depth sensing errors, the system may still experience trajectory disturbances and frequent local path re-planning, indicating that further optimization is needed to handle complex perceptual uncertainties effectively.

In summary, the proposed multi-level path planning system demonstrates robust planning success rates and effective obstacle avoidance performance in complex dynamic environments, providing strong support for practical deployment of robotic arms. The system offers engineering applicability through its integrated architecture, perception fusion, policy expressiveness, and robustness. Future work will focus on multi-objective task planning, adaptation to complex scenarios, and real-world validation, aiming to advance the system's practical deployment in industrial settings. In further research, the algorithm can be extended to real robotic platforms, integrating real-time perception systems to advance perception-control integration. Future work may also incorporate architectures such as Graph Neural Networks (GNNs) and Transformers to enhance adaptability in unknown environments, and explore multimodal perception and collaborative planning strategies, thereby further advancing the practical applications of this research in industrial, logistics, medical, and other domains.



(a) Variation Curves of Robotic Arm Joint Angles with End-Effector Coordinates



(b) Variation Curves of Robotic Arm Joint Angles with End-Effector Coordinates

Fig. 13. Comparison of Six-Joint Angle Variation Trends of the Robotic Arm During Path Execution.

CRediT authorship contribution statement

Qi Sun: Writing – review & editing, Writing – original draft, Project administration, Methodology, Conceptualization. **Jianhao Guo:** Visualization, Validation, Software, Data curation. **Guobing Sun:** Supervision.

Declaration of competing interest

The authors declare that they have no competing financial interests.

Data availability

Data will be made available on request.

References

- [1] A. Abdi, D. Adhikari, J.H. Park, A novel hybrid path planning method based on q-learning and neural network for robot arm, *Appl. Sci.* 11 (2021) 6770.
- [2] M. Al-Gabalawy, Path planning of robotic arm based on deep reinforcement learning algorithm, *Adv. Control Appl. Eng. Indust. Syst.* 4 (2022) e79.
- [3] S.J. Al-Kamil, R. Szabolcsi, Optimizing path planning in mobile robot systems using motion capture technology, *Results Eng.* 22 (2024) 102043.
- [4] B.B.K. Ayawli, R. Chellali, A.Y. Appiah, F. Kyeremeh, An overview of nature-inspired, conventional, and hybrid methods of autonomous vehicle path planning, *J. Adv. Transp.* 2018 (2018) 8269698.
- [5] H. Bae, G. Kim, J. Kim, D. Qian, S. Lee, Multi-robot path planning method using reinforcement learning, *Appl. Sci.* 9 (2019) 3057.
- [6] H.K. Banga, P. Kalra, R. Kumar, S. Singh, C.I. Pruncu, Optimization of the cycle time of robotics resistance spot welding for automotive applications, *J. Adv. Manuf. Proc.* 3 (2021) e10084.

- [7] C. Cadena, L. Carlone, H. Carrillo, Y. Latif, D. Scaramuzza, J. Neira, I. Reid, J.J. Leonard, Past, present, and future of simultaneous localization and mapping: toward the robust-perception age, *IEEE Trans. Robot.* 32 (2016) 1309–1332.
- [8] A. Candra, M.A. Budiman, K. Hartanto, Dijkstra's and a-star in finding the shortest path: a tutorial, in: 2020 International Conference on Data Science, Artificial Intelligence, and Business Analytics (DATABIA), IEEE, 2020, pp. 28–32.
- [9] Y. Chen, L. Liang, Slp-improved ddpq path-planning algorithm for mobile robot in large-scale dynamic environment, *Sensors* 23 (2023) 3521.
- [10] C. Cui, Z. Wang, J. Sui, Y. Zhang, C. Guo, An improved rrt behavioral planning method for robots based on ptm algorithm, *Sci. Rep.* 14 (2024) 21776.
- [11] J. Ding, Y. Zhou, X. Huang, K. Song, S. Lu, L. Wang, An improved rrt* algorithm for robot path planning based on path expansion heuristic sampling, *J. Comput. Sci.* 67 (2023) 101937.
- [12] S. El Ghazouali, Y. Mhirit, A. Oukhrd, U. Michelucci, H. Nouira, Fusionvision: a comprehensive approach of 3d object reconstruction and segmentation from rgb-d cameras using yolo and fast segment anything, *Sensors* 24 (2024) 2889.
- [13] Y. Gu, Z. Zhu, J. Lv, L. Shi, Z. Hou, S. Xu, Dm-dqn: dueling munchausen deep q network for robot path planning, *Complex Intell. Syst.* 9 (2023) 4287–4300.
- [14] H. Honari, S. Khodaygan, R3t*-mosafer! (backslash lambda): path planning of mobile robots in unknown dynamic environments, *Intell. Serv. Robot.* 17 (2024) 1175–1188.
- [15] F. Huo, S. Zhu, H. Dong, W. Ren, A new approach to smooth path planning of ackerman mobile robot based on improved aco algorithm and b-spline curve, *Robot. Auton. Syst.* 175 (2024) 104655.
- [16] A.T. Khan, S. Li, S. Kadry, Y. Nam, Control framework for trajectory planning of soft manipulator using optimized rrt algorithm, *IEEE Access* 8 (2020) 171730–171743.
- [17] H. Le, S. Saeedvand, C.C. Hsu, A comprehensive review of mobile robot navigation using deep reinforcement learning algorithms in crowded environments, *J. Intell. Robot. Syst.* 110 (2024) 1–22.
- [18] X. Lei, Z. Zhang, P. Dong, Dynamic path planning of unknown environment based on deep reinforcement learning, *J. Robot.* 2018 (2018) 5781591.
- [19] C. Liang, L. Liu, C. Liu, Multi-uav autonomous collision avoidance based on ppo-gic algorithm with cnn-lstm fusion network, *Neural Netw.* 162 (2023) 21–33.
- [20] T. Linder, K.Y. Pfeiffer, N. Vaskevicius, R. Schirmer, K.O. Arras, Accurate detection and 3d localization of humans using a novel yolo-based rgb-d fusion approach and synthetic training data, in: 2020 IEEE International Conference on Robotics and Automation (ICRA), IEEE, 2020, pp. 1000–1006.
- [21] A. Marashian, A. Razminia, Mobile robot's path-planning and path-tracking in static and dynamic environments: dynamic programming approach, *Robot. Auton. Syst.* 172 (2024) 104592.
- [22] H. Mazaheri, S. Goli, A. Nourollah, Path planning in three-dimensional space based on butterfly optimization algorithm, *Sci. Rep.* 14 (2024) 2332.
- [23] A. Nagabandi, I. Clavera, S. Liu, R.S. Fearing, P. Abbeel, S. Levine, C. Finn, Learning to adapt in dynamic, real-world environments through meta-reinforcement learning, *arXiv preprint arXiv:1803.11347*, 2018.
- [24] A. Ozcan, O. Cetin, A novel fusion method with thermal and rgb-d sensor data for human detection, *IEEE Access* 10 (2022) 66831–66843.
- [25] N. Promkaew, S. Thammawiset, P. Srisan, P. Sanitchon, T. Tummawai, S. Sukpancharoen, Development of metaheuristic algorithms for efficient path planning of autonomous mobile robots in indoor environments, *Results Eng.* 22 (2024) 102280.
- [26] H. Ravichandar, A.S. Polydoros, S. Chernova, A. Billard, Recent advances in robot learning from demonstration, *Annu. Rev. Control Robot. Auton. Syst.* 3 (2020) 297–330.
- [27] A. Scalvini, A. Suarez, S.R. Nekoo, A. Ollero, Finite-time state-dependent Riccati equation regulation of anthropomorphic dual-arm space manipulator system in free-flying conditions, *Acta Astronaut.* 216 (2024) 504–517.
- [28] R. Senthilnathan, et al., Quicker path planning of a collaborative dual-arm robot using modified bp-rrt* algorithm, *Int. J. Comput. Commun. Control* 19 (2024).
- [29] G. Sharma, Pros and cons of different sampling techniques, *Int. J. Appl. Res.* 3 (2017) 749–752.
- [30] S. Shokouhi, O. Oruc, M.W. Thein, Self-supervised learning-based path planning and obstacle avoidance using ppo and b-splines in unknown environments, *arXiv preprint arXiv:2412.02176*, 2024.
- [31] B. Siciliano, O. Khatib, Robotics and the handbook, in: *Springer Handbook of Robotics*, Springer, 2016, pp. 1–6.
- [32] M. Takahashi, Y. Ji, K. Umeda, A. Moro, Expandable yolo: 3d object detection from rgb-d images, in: 2020 21st International Conference on Research and Education in Mechatronics (REM), IEEE, 2020, pp. 1–5.
- [33] H. Wang, S. Lou, J. Jing, Y. Wang, W. Liu, T. Liu, The ebs-a* algorithm: an improved a* algorithm for path planning, *PLoS ONE* 17 (2022) e0263841.
- [34] Z. Wang, Y. Chen, H. Ding, Z. Liu, Y. Xie, K. Zhou, P. Feng, Y. Yu, C. Li, D. Zhang, et al., Yolov8-dbw: an improved high-accuracy fast landslide detection model, *Trans. GIS* 29 (2025) e70021.
- [35] M. Yang, J. Liu, Research on six-degree-of-freedom refueling robotic arm positioning and docking based on rgb-d visual guidance, *Appl. Sci.* 14 (2024) 4904.
- [36] J. Zhang, Z. Wang, J. Lai, H. Wang, Gptarm: an autonomous task planning manipulator grasping system based on vision-language models, *Machines* 13 (2025) 247.
- [37] X. Zhou, J. Yan, M. Yan, K. Mao, R. Yang, W. Liu, Path planning of rail-mounted logistics robots based on the improved Dijkstra algorithm, *Appl. Sci.* 13 (2023) 9955.
- [38] G. Zhu, Z. Shen, L. Liu, S. Zhao, F. Ji, Z. Ju, J. Sun, Auv dynamic obstacle avoidance method based on improved ppo algorithm, *IEEE Access* 10 (2022) 121340–121351.